

CAHIER DES CHARGES

AURA

Assistant Universel Réactif et Autonome

CAHIER DES CHARGES — VERSION 0.4

Recentrage technique & intégration d'AXIS — succède à la V0.3.2 (+ addenda V0.3.3)

Document de référence unique — Septembre 2026

Porteur de projet : L

Note sur cette édition

La V0.4 recentre AURA sur les domaines informatiques et techniques et retire entièrement le périmètre Gaming & Streaming (§8 de la V0.3.2, supprimé sans remplacement). Elle introduit l'intégration d'AXIS — un environnement de commande générique, indépendant et réutilisable, développé comme projet à part entière plutôt que comme fonctionnalité propriétaire d'AURA. AURA devient l'un de ses utilisateurs, via un module dédié qui expose ses propres capacités sans que le cœur d'AXIS ne connaisse AURA. Le détail des changements par rapport à la V0.3.2 figure en section 24.

Sommaire

- 1 Présentation du projet
- 2 Objectifs
- 3 Périmètre fonctionnel complet
- 4 Exigences fonctionnelles — Fondations
- 5 Exigences fonctionnelles — Extension JARVIS + The Machine
- 6 AXIS — Terminal générique, indépendant, et son intégration à AURA
- 7 Connecteur Développement
- 8 Connecteur Productivité
- 9 Connecteur Cartographie
- 10 Vision avancée & AURA Image Enhancer
- 11 Extensions transverses et agents
- 12 Modèle de données
- 13 Architecture technique
- 14 Système de permissions
- 15 Contrats de connecteurs — synthèse étendue
- 16 Maquettes et parcours utilisateur
- 17 Exigences non fonctionnelles
- 18 Contraintes, limites et principes de sécurité
- 19 Scénarios de test et critères d'acceptation
- 20 Feuille de route et estimation de charge
- 21 Budget prévisionnel
- 22 Critères de succès
- 23 Risques et mitigation
- 24 Historique des versions
- 25 Annexe — Glossaire

1. Présentation du projet

1.1 Contexte

AURA est un projet personnel porté par L, développeur et créateur autodidacte actif sur plusieurs fronts techniques : développement web, développement logiciel, modélisation 3D et design graphique. Le projet a démarré comme un assistant modulaire (V0.2), puis s'est étendu selon une trajectoire inspirée de JARVIS (Iron Man) et de la Machine (Person of Interest) — voix, vision, supervision système, sécurité défensive et autonomie contrôlée (V0.3), avant de recevoir un sous-module d'amélioration d'image (V0.3.1) et l'intégration d'une carte mentale de capacités (V0.3.2).

Changement V0.4 — Le périmètre Gaming & Streaming, présent depuis la toute première version (V0.2), est intégralement retiré. AURA se recentre sur les domaines informatiques et techniques — développement, cybersécurité, administration système et réseau, automatisation, analyse de données, IA — et intègre AXIS, un environnement de commande générique, indépendant et réutilisable, détaillé en section 6. AXIS n'est pas développé pour AURA : c'est un projet à part entière qu'AURA consomme via un module dédié, au même titre qu'un autre projet futur pourrait le faire.

1.2 Vision

« Une IA capable de tout faire, sur tout support. » — *formulation initiale du besoin.*

Cette formulation reste la boussole du projet, traduite en système réel et réalisable : une intelligence artificielle personnelle, modulaire et extensible, construite autour d'un cœur IA et d'un réseau de connecteurs et d'agents contrôlés. L'ambition est de faire évoluer AURA d'un assistant modulaire vers un véritable centre de contrôle technique personnel : conversation, mémoire, vision, commande directe via AXIS, analyse de données, automatisation, développement, supervision du poste, agents spécialisés et centre d'opérations.

« Omnipotence » désigne ici une trajectoire d'extension continue, jamais un accès illimité de fait : chaque capacité dépend d'un connecteur, d'une autorisation et d'un système réellement accessible (voir section 18) — que l'accès passe par la conversation ou par AXIS.

2. Objectifs

2.1 Objectif général

Concevoir et développer AURA, un assistant IA personnel unique, capable d'orchestrer un nombre croissant de domaines informatiques et techniques — développement, cybersécurité, administration système et réseau, automatisation, analyse de données, IA, productivité — via une architecture modulaire, avec une mémoire persistante, une intégration au Terminal générique AXIS et une identité visuelle et conversationnelle cohérente.

2.2 Objectifs spécifiques

- Centraliser dans une seule interface les outils déjà développés et les futurs modules.

- Donner à AURA une mémoire à long terme : préférences, projets en cours, historique des échanges, événements et connaissances.
- Permettre à AURA d'agir concrètement (exécuter des tâches, appeler des API, contrôler des applications) et pas seulement de répondre.
- **Intégrer AXIS, un Terminal générique et indépendant** — un langage de commande et de script qui n'est pas spécifique à AURA, permettant d'accéder directement aux capacités d'AURA (via son propre module AXIS) sans détour par la conversation, pour les usages où la précision et la répétabilité priment sur le langage naturel. AXIS reste utilisable et réutilisable en dehors d'AURA (§6.8).
- Construire une architecture ouverte, où chaque nouvelle capacité s'ajoute sous forme de module, connecteur ou agent sans réécrire le cœur du système — ni celui d'AURA CORE, ni celui d'AXIS, qui reste lui-même indépendant du module qui l'utilise.
- Conserver à tout moment un contrôle humain sur les actions sensibles, quelle que soit l'interface utilisée pour les déclencher.
- Développer une identité de marque propre à AURA — fond noir, accents rouges, esthétique HUD sombre et futuriste.
- Permettre l'affichage cartographique interactif dans le même esprit visuel « centre d'opérations », y compris comme outil de visualisation technique (infrastructure, sécurité).

3. Périmètre fonctionnel complet

AURA est un ensemble de modules et d'agents indépendants reliés à un cœur commun. Chaque ligne indique la priorité fixée et l'état réel d'avancement.

Groupe A — Fondations (V0.2)

Module	Rôle	Priorité	Statut
Assistant & productivité	Notes, tâches, rappels, résumés, rédaction, recherche.	Haute	Codé
Développement & code	Aide au code, revue, debug, Git.	Haute	Codé
Création	Assets visuels, design UI/HUD.	Moyenne	Spécifié
Domotique / IoT	Contrôle d'appareils connectés.	Basse	Spécifié
Communication	Rédaction et suivi de messages mail/Discord.	Moyenne	Spécifié
Cartographie interactive	Carte 2D façon centre d'opérations.	Moyenne	Spécifié
Mémoire & apprentissage	Historique, préférences, personnalisation continue.	Haute	Codé

~~Gaming & streaming~~ — retiré du périmètre en V0.4 (§1.1, §24). Aucune ligne de remplacement : le retrait est complet, pas une simple dépriorisation.

Groupe B — Extension JARVIS + The Machine (V0.3) + AXIS (V0.4)

Module	Rôle	Priorité	Statut
--------	------	----------	--------

AURA CORE	Cœur décisionnel : raisonnement, planification, routage.	Très haute	Partiel
AXIS (intégration)	Terminal générique, indépendant et réutilisable, intégré à AURA via un module dédié qui expose ses capacités sans que le cœur d'AXIS ne connaisse AURA (détail §6).	Très haute	Spécifié
AURA VOICE	Wake word, reconnaissance et synthèse vocale.	Haute	Spécifié
AURA VISION	Analyse d'écran, caméra, images, OCR.	Haute	Spécifié
AURA OS	Contrôle autorisé du poste : apps, fichiers, scripts, réseau.	Haute	Spécifié
AURA ANALYTICS	Analyse de données, anomalies, prévisions probabilistes, IA/ML.	Haute	Spécifié
AURA AGENTS	Réseau de 10 agents spécialisés, reliés en toile (détail §5.6).	Haute	Spécifié
AURA SYSTEM MONITOR	Surveillance CPU, GPU, RAM, réseau.	Haute	Spécifié
AURA SECURITY	Sécurité défensive et pentest encadré — pilier du recentrage V0.4.	Très haute	Spécifié
AURA AUTONOMY	Règles, tâches planifiées, actions sûres automatisées.	Haute	Spécifié
AURA WORLD	Centre d'opérations : carte, statuts, événements.	Moyenne	Spécifié

Groupe C — Sous-modules et extensions

Module	Rôle	Priorité	Statut
AURA IMAGE ENHANCER	Sous-module d'AURA VISION : upscaling, restauration, netteté.	Haute	Spécifié
AURA EDUCATION	Pédagogie technique, tutorat développement/sécurité, traduction.	Moyenne	Spécifié
AURA OFFICE	Documents techniques, tableurs, rapports, PDF.	Haute	Spécifié

« Codé » : testé et fonctionnel dans le squelette applicatif de référence. « Partiel » : la version de base existe (cœur conversationnel) mais pas les capacités avancées de la section 5. « Spécifié » : prévu ici, pas encore implémenté.

4. Exigences fonctionnelles — Fondations (V0.2)

4.1 Cœur conversationnel

ID	Exigence	Priorité
F-01	AURA doit comprendre des requêtes en langage naturel (français en priorité) et y répondre de manière cohérente.	Haute

F-02	AURA doit conserver le contexte d'une conversation et le fil des demandes en cours.	Haute
F-03	AURA doit pouvoir déléguer une requête au bon module (routage d'intention) sans intervention manuelle.	Haute
F-04	AURA doit pouvoir exprimer ses limites plutôt que d'inventer une réponse lorsqu'elle ne sait pas ou ne peut pas agir.	Haute
F-20	Le ton d'AURA doit s'adapter au contexte d'usage : rigoureux et précis pour le développement et la sécurité, plus pédagogique pour l'apprentissage (AURA EDUCATION), neutre et factuel dans AXIS.	Moyenne

F-20 révisée en V0.4 : la référence au gaming/streaming est retirée, remplacée par les registres de ton réellement utiles au nouveau périmètre.

4.2 Mémoire et personnalisation

ID	Exigence	Priorité
F-05	AURA doit mémoriser les préférences explicites de l'utilisateur (style, outils, projets en cours).	Haute
F-06	L'utilisateur doit pouvoir consulter, corriger ou effacer ce qu'AURA a mémorisé.	Haute
F-07	AURA doit pouvoir relier une nouvelle demande à un projet déjà connu.	Moyenne

4.3 Action et automatisation

ID	Exigence	Priorité
F-08	AURA doit pouvoir exécuter des actions techniques (scripts, appels API) via des connecteurs dédiés.	Haute
F-09	Toute action à conséquence réelle (envoi, achat, suppression, modification externe) doit être confirmée par l'utilisateur, qu'elle soit déclenchée en conversation ou depuis AXIS.	Haute
F-10	AURA doit journaliser les actions effectuées pour permettre un contrôle a posteriori.	Moyenne
F-11	AURA doit pouvoir planifier des tâches récurrentes.	Moyenne

4.4 Interfaces

ID	Exigence	Priorité
F-12	AURA doit être invisible en dehors d'une session active : aucune icône permanente, aucun raccourci. Le seul point d'entrée est une commande tapée dans un terminal système, qui ouvre une fenêtre d'application dédiée — cette fenêtre est AURA.	Haute
F-13	AURA doit être accessible en complément depuis Discord.	Moyenne
F-14	Une entrée vocale locale est envisageable en phase ultérieure (voir AURA VOICE, §5.2).	Basse

F-21	La fenêtre d'application doit afficher un rendu en réseau de nœuds connectés (toile) : branches principales (agents) et secondaires (leurs outils), reliées entre elles comme les fils d'une toile d'araignée — dans l'esprit de The Machine (Person of Interest), sur la palette noir/rouge déjà définie. La zone de conversation et la toile partagent le même écran.	Haute
F-22	Aucun processus AURA ne doit s'exécuter tant que la commande de lancement n'a pas été invoquée : ni service, ni tâche planifiée, ni icône en barre des tâches.	Haute
F-23	AURA doit intégrer AXIS — un environnement de commande générique et indépendant, avec son propre langage — comme point d'accès direct à ses capacités (via le module AURA pour AXIS), complémentaire à la conversation en langage naturel (détail §6).	Très haute
F-24	Toute commande AXIS qui invoque le module AURA doit respecter les mêmes règles de sensibilité et de permission que les autres interfaces d'AURA (§14) — aucun contournement, et aucun accès à un interpréteur système générique par ce biais.	Haute
F-25	AXIS doit permettre l'écriture, la sauvegarde et l'exécution de scripts (extension .axs) combinant plusieurs commandes, pour l'automatisation de tâches répétitives — quel que soit le module chargé (détail §6.5).	Haute

Il n'existe aucune interface web : la fenêtre d'application (§13.2) est l'unique surface d'AURA, y compris pour l'Image Lab (§10.2), la Cartographie (§9) et AXIS (§6), qui vivent comme des écrans internes de cette même fenêtre plutôt que comme des pages séparées. La surveillance d'AURA AUTONOMY (§5.9) n'est active que pendant qu'une session est ouverte, conformément à F-22.

4.5 Cartographie

ID	Exigence	Priorité
F-15	AURA doit pouvoir zoomer vers un pays, une ville ou un lieu précis à la demande.	Haute
F-16	La carte doit permettre le zoom et le déplacement, et afficher des repères statiques ou dynamiques.	Moyenne
F-17	Le rendu de la carte doit respecter l'identité visuelle sombre et futuriste d'AURA.	Haute
F-18	La carte doit s'afficher uniquement en vue plane 2D, à l'exclusion de tout globe 3D.	Haute
F-19	Par défaut la carte affiche une vue mondiale ; elle ne zoome que sur demande explicite.	Haute

5. Exigences fonctionnelles — Extension JARVIS + The Machine (V0.3)

AURA doit évoluer selon le principe : observer → comprendre → analyser → planifier → proposer → agir → vérifier. L'objectif n'est pas de donner à AURA un accès illimité, mais de lui permettre d'acquérir progressivement de nouvelles capacités via des connecteurs contrôlés — que la demande vienne de la conversation ou d'AXIS (§6).

5.1 AURA CORE — cœur IA avancé

- Compréhension du langage naturel et du contexte.
- Raisonnement multi-étapes et décomposition automatique des demandes.
- Planification de tâches complexes sous forme de plans exécutables.
- Routage automatique vers un ou plusieurs modules, coordination de plusieurs agents.
- Vérification du résultat, détection d'échec et réévaluation du plan.
- Gestion des priorités et des tâches concurrentes.
- Refus explicite lorsqu'une capacité ou une autorisation manque.

5.2 AURA VOICE — interface vocale

- Wake word « AURA » configurable.
- Speech-to-Text local ou via fournisseur choisi ; Text-to-Speech avec voix configurable.
- Détection des interruptions et reprise de conversation ; mode mains libres.
- Retour vocal prioritaire pour les alertes importantes.
- Possibilité de désactiver totalement le microphone.

5.3 AURA VISION — vision artificielle

- Capture contrôlée de l'écran ; analyse d'images et de captures ; OCR.
- Reconnaissance d'objets, d'éléments d'interface, de scènes et de changements visuels.
- Détection de fenêtres, erreurs et notifications.
- Analyse caméra uniquement après activation explicite ; historique d'observation limité.
- Sous-module AURA IMAGE ENHANCER détaillé en section 10.

5.4 AURA OS — agent informatique

- Lancer et fermer des applications autorisées.
- Lire, organiser, créer et modifier des fichiers dans les espaces autorisés.
- Exécuter des scripts et commandes explicitement autorisés ; surveiller les processus.
- Automatiser des workflows répétitifs ; contrôler des outils de développement (Git, Node.js, gestionnaires de paquets, environnements virtuels/conteneurs).
- **Administration système et réseau (nouveau, V0.4)** : gestion des services locaux, interfaces réseau, diagnostics (ping, traceroute, résolution DNS), consultation et configuration du pare-feu local — toujours dans les espaces et systèmes autorisés.
- Navigation web autonome pour le compte de l'utilisateur, sous les mêmes règles de confirmation.
- Présenter un aperçu avant toute action externe sensible.

5.5 AURA ANALYTICS — moteur d'analyse et d'apprentissage automatique

- Fusion de données provenant de plusieurs connecteurs ; analyse statistique et temporelle.
- Détection d'anomalies et de tendances ; recherche de corrélations ; scoring et classement.
- Simulation de scénarios ; prévisions probabilistes avec niveau de confiance.
- **Modèles prédictifs et apprentissage automatique (nouveau, V0.4)** appliqués aux données observées par AURA — classification, détection d'anomalies apprise plutôt que seulement fondée sur des règles, amélioration progressive à partir de l'historique.

- Traçabilité de la provenance des données ; ne jamais présenter une prévision comme une certitude.

5.6 AURA AGENTS — architecture multi-agents

Réseau d'agents spécialisés délégués par AURA CORE. Au-delà de son lien avec AURA CORE, chaque agent peut aussi être relié directement à d'autres agents — ce sont ces connexions transversales qui donnent à la toile de la fenêtre applicative (§16.1, F-21) la forme d'un véritable réseau plutôt que d'une simple étoile. La carte de ces relations figure en §5.6.1.

Changement V0.4 — AURA GAME et AURA STREAM sont supprimés sans remplacement (périmètre Gaming & Streaming retiré, §1.1). Le réseau passe de 12 à 10 agents. Plutôt que d'ajouter un agent par domaine technique cité en objectif (§2), les domaines qui recoupaient un agent existant l'étoffent (AURA CODE, AURA ANALYTICS, AURA OS) : ça évite d'ajouter des modules pour le seul effet d'en augmenter le nombre, contraire à l'exigence de cohérence du projet.

- AURA CODE — programmation, revue, debug, tests, applications web et mobiles. **Étoffé (V0.4)** : analyse de code statique (qualité, style, complexité) et gestion des environnements de développement (dépendances, environnements virtuels, conteneurs).
- AURA DATA — traitement et visualisation de données.
- AURA SYSTEM — poste, processus et performances.
- AURA SECURITY — sécurité défensive et pentest encadré des systèmes autorisés (détail §5.8).
- AURA CREATIVE — design, 3D, assets, création et génération d'images.
- AURA RESEARCH — recherche, synthèse documentaire et vérification de faits.
- AURA HOME — domotique et appareils connectés.
- AURA PRODUCTIVITY — tâches, rappels, projets et organisation.
- AURA EDUCATION — pédagogie technique (apprentissage du code, de la sécurité, des systèmes), tutorat personnalisé, traduction.
- AURA OFFICE — documents bureautiques et techniques, tableurs, présentations, PDF.

5.6.1 Carte des relations entre agents

Chaque fil de la toile correspond à une relation réelle, pas à un motif décoratif : soit un agent consomme les données d'un autre, soit il déclenche ou complète son action. AURA WORLD est relié à tous les agents par nature (il affiche leur état, §5.10) et n'est donc pas répété ci-dessous. AXIS (§6) n'est pas un agent AURA : c'est un Terminal générique et indépendant, dont AURA se sert via son propre module pour atteindre le même registre de connecteurs que tous les agents ci-dessous exposent — voir §6.1 et §6.7.

Agent	Relié à	Nature de la relation
AURA AUTONOMY	AURA SYSTEM MONITOR	Déclencheurs sur seuils et anomalies système (§5.9).
AURA AUTONOMY	AURA SECURITY	Escalade automatique vers l'utilisateur sur activité inhabituelle détectée.
AURA AUTONOMY	AURA HOME	Automatisations de la maison/bureau déclenchées par des règles.
AURA AUTONOMY	Module AURA pour AXIS	Une règle peut déclencher l'exécution d'un script AXIS (.axs, §6.5, §6.9).

AURA ANALYTICS	AURA DATA	Fusion et visualisation des données traitées.
AURA ANALYTICS	AURA SYSTEM MONITOR	Détection d'anomalies à partir des métriques système.
AURA SECURITY	AURA CODE	Audit des dépendances et des secrets exposés dans les projets (§5.8).
AURA SECURITY	AURA SYSTEM	Surveillance des événements système autorisés.
AURA CREATIVE	AURA VISION	Partage du connecteur Vision pour la génération et l'amélioration d'images (§10, §11.1).
AURA RESEARCH	AURA OFFICE	Rédaction de documents à partir de résultats de recherche vérifiés.
AURA EDUCATION	AURA RESEARCH	Le tutorat s'appuie sur la recherche et la vérification de faits (§11.1).

5.7 AURA SYSTEM MONITOR — supervision du poste

- CPU, GPU, RAM : utilisation, fréquence, température et mémoire lorsqu'elles sont accessibles.
- Stockage : espace, activité, état SMART si disponible ; réseau : latence, débit, pertes.
- Processus : consommation et comportements anormaux.
- Détection d'écarts par rapport aux habitudes enregistrées ; alertes configurables.

5.8 AURA SECURITY — sécurité défensive et pentest encadré

Pilier du recentrage V0.4 — avec le retrait du Gaming & Streaming, la cybersécurité devient l'un des deux domaines les plus prioritaires du projet aux côtés du développement (§2, §3). Le module ne change pas de forme mais gagne en priorité (Très haute, §3).

Le module couvre deux volets : un audit défensif et un test d'intrusion offensif encadré. Les deux restent strictement bornés aux systèmes autorisés.

- Surveillance des événements système autorisés ; audit de configuration.
- Analyse des dépendances des projets ; détection de secrets accidentellement exposés.
- Analyse de journaux, détection d'activités inhabituelles, génération de rapports de sécurité.
- Test d'intrusion offensif — autorisé uniquement sur les systèmes propres de l'utilisateur, des environnements CTF, ou des cibles couvertes par une autorisation écrite explicite (bug bounty).
- Actions de remédiation uniquement dans les systèmes autorisés.
- Aucune fonction destinée à contourner une protection, compromettre un tiers non autorisé ou automatiser une action malveillante.

5.9 AURA AUTONOMY — autonomie contrôlée

- Surveillance continue des événements autorisés, tant qu'une session est ouverte (F-22 : aucun processus ne persiste hors session).
- Déclencheurs : horaire, événement, seuil, changement de fichier ou état d'application.
- Tâches récurrentes et planification de workflows.

- Actions automatiques uniquement pour les opérations classées sûres ; escalade vers l'utilisateur sinon.
- **Une règle peut déclencher un script AXIS (.axs, §6.5)** qui invoque le module AURA, comme action au même titre qu'une action de connecteur unique — un workflow complexe peut ainsi être écrit une fois dans AXIS puis réutilisé comme automatisation.
- Arrêt d'urgence global ; journal complet des décisions et actions.
- Mode simulation permettant de montrer ce qu'AURA ferait sans l'exécuter.

5.10 AURA WORLD — centre d'opérations

- Carte mondiale 2D par défaut, conforme à l'exigence F-19.
- Zoom vers pays, villes et lieux ; marqueurs statiques ou dynamiques.
- Affichage de l'état des agents et connecteurs ; vue des tâches actives.
- Flux d'événements, alertes et notifications ; statistiques système.
- **Usage technique (V0.4)** : géolocalisation de repères en contexte de sécurité (ex. origine d'événements réseau suspects détectés par AURA SECURITY) et visualisation d'infrastructure distribuée — au-delà de sa fonction de tableau de bord général.
- Interface HUD sombre, accents rouges, esthétique futuriste.

6. AXIS — Terminal générique, indépendant, et son intégration à AURA

Composant fondamental, mais pas propriétaire (V0.4) — AXIS est l'un des deux points d'entrée d'AURA, au même titre que la conversation en langage naturel (§4.1). Mais AXIS n'est **pas développé pour AURA** : c'est un projet indépendant, à vocation réutilisable, qu'AURA consomme comme n'importe quel autre projet pourrait le faire. Ce que la section 5 appelait « AURA TERMINAL » se scinde donc en deux : **AXIS** (le cœur générique, §6.1 à §6.6) et le **module AURA pour AXIS** (l'intégration spécifique à ce projet, §6.7 à §6.12).

6.1 Positionnement : AXIS comme projet indépendant

La conversation en langage naturel est excellente pour une demande ambiguë ou exploratoire, mais imprécise et lente à répéter pour un geste technique connu — lancer un scan de sécurité sur un projet donné, enchaîner trois commandes Git, ou relancer chaque matin le même diagnostic système. AXIS répond à ce besoin : un langage court, déterministe et scriptable, pour l'utilisateur qui sait déjà ce qu'il veut faire — dans n'importe quel projet, pas seulement AURA.

AXIS se développe et se maintient comme un module logiciel séparé (paquet Node.js autonome, dépôt propre), sans dépendance vers le code d'AURA. Il ne connaît ni AURA CORE, ni les connecteurs d'AURA, ni sa mémoire, ni son identité visuelle. Sa seule responsabilité : interpréter un langage de commande générique et l'exécuter contre des **modules** qui s'y enregistrent (§6.4) — quel que soit le projet qui les fournit. AURA est le premier consommateur d'AXIS, pas son propriétaire.

Conséquence directe : AXIS peut s'utiliser seul, en ligne de commande classique, sans AURA ni interface graphique — voir §6.8 (réutilisation dans d'autres projets).

6.2 Architecture en deux couches

Tout le reste de cette section applique un principe unique : séparer ce qui est générique (réutilisable par n'importe quel projet) de ce qui est spécifique à AURA.

```

AXIS CORE (projet independant)
├─ Parseur du langage (§6.3)
├─ Moteur d'execution (pipes, variables, if/for)
├─ Registre de modules (§6.4) - ne connait aucun module a l'avance
├─ Point d'interception generique (§6.6) - hook, pas de politique integree
├─ CLI autonome - utilisable hors de toute application graphique
│
│   └─ enregistre un module via l'API generique du registre
│       ▼
MODULE AURA POUR AXIS (§6.7, specifique a ce projet)
├─ Expose les connecteurs deja definis en §7 a §11 (git, task, security, ...)
├─ Implemente le hook d'interception avec le Policy Engine d'AURA (§14)
├─ Journalise chaque commande dans le meme Audit Logger qu'AURA CORE (§13.1)

```

Un autre projet brancherait son propre module à la place (ou en plus) du module AURA, sans toucher au cœur d'AXIS — c'est précisément ce qui rend AXIS réutilisable (§6.8).

6.3 Le langage de commande (générique)

Le langage lui-même ne mentionne jamais AURA : il cible des actions au format `espace.action`, où `espace` désigne un module enregistré — qu'il vienne d'AURA ou d'un autre projet.

```

# Forme de base : espace.action --parametre valeur
git.status
git.commit --message "correctif auth"
task.create --title "Relire le rapport" --priority haute --due 2026-09-15

# Les guillemets ne sont necessaires que si la valeur contient un espace
security.scan_logs --source auth.log --since 24h

```

6.3.1 Enchaînement et flux de données (pipes)

La sortie d'une commande peut être transmise à la suivante avec `|`, dans l'esprit d'un shell Unix mais limité aux actions enregistrées par un module :

```

system.metrics | analytics.detect_anomaly
security.scan_logs --source auth.log | analytics.detect_anomaly --sensitivity haute |
alert.notify

```

6.3.2 Variables

```

$rapport = security.audit_project --path ./aura
show $rapport.summary
task.create --title "Corriger : " + $rapport.top_issue --priority haute

```

6.3.3 Structures de contrôle

Un jeu minimal de structures suffisant à des workflows réels, sans viser un langage de programmation généraliste :

```
if $rapport.severity == "critique" then
    alert.notify --message "Vulnerabilite critique detectee"
    task.create --title "Traiter l'alerte securite" --priority haute
end

for $tache in task.list --status active
    if $tache.overdue then
        reminder.schedule --text $tache.title --at "+1h"
    end
end
```

6.3.4 Alias

```
alias secu-quick = security.scan_logs --source auth.log --since 24h
# s'utilise ensuite comme n'importe quelle commande :
secu-quick
```

6.4 Système de modules et d'extensions AXIS

Un module AXIS est un paquet indépendant qui s'enregistre auprès du cœur au démarrage, en déclarant un manifeste : les espaces de commandes qu'il expose, leurs actions, leurs paramètres, et — s'il le souhaite — un gestionnaire d'interception (§6.6).

```
// Forme simplifiée du manifeste qu'un module déclare à AXIS CORE
module.register({
    namespace: "git",
    actions: {
        status: { sensitivity: "read", handler: gitStatusHandler },
        commit: { sensitivity: "reversible", params: ["message"], handler: gitCommitHandler }
    },
    onBeforeExecute: /* optionnel : hook d'interception du module (§6.6) */
})
```

AXIS CORE ne connaît que cette interface : il ne sait pas ce qu'est « git » ni « task », seulement qu'un module a déclaré un espace de ce nom avec ces actions. Ajouter, retirer ou mettre à jour un module ne touche jamais au parseur ni au moteur d'exécution — c'est la condition de l'évolutivité à long terme visée par cette révision (§6.11).

6.5 Scripts AXIS (.axs)

Un script est un fichier texte .axs (*AXIS Script* — extension neutre, sans référence à AURA) contenant une suite de commandes du langage ci-dessus, sauvegardé, réutilisable et versionnable comme n'importe quel fichier de code. Il s'exécute avec `run mon_script.axs`, quels que soient les modules chargés.

```
## diagnostic-matin.axs (utilise ici le module AURA, mais le format est generique)
# Diagnostic quotidien : etat systeme + verification securite de base

$etat = system.metrics
if $etat.cpu > 85 then
    alert.notify --message "CPU eleve au demarrage"
end

git.status
security.scan_logs --source auth.log --since 24h

show "Diagnostic termine."
```

Un script peut être :

- lancé manuellement depuis AXIS (run diagnostic-matin.axs), dans AURA ou en ligne de commande autonome ;
- déclenché par une règle AURA AUTONOMY comme action planifiée (§5.9), si le module AURA est chargé ;
- partagé et réutilisé entre projets, dès lors que les modules qu'il invoque sont disponibles — un script qui n'utilise que des commandes génériques (ex. définies par un futur module « fichiers ») reste portable tel quel.

6.6 Gouvernance et sécurité — un point d'interception générique

AXIS CORE n'impose **aucune politique de permission** : il ne sait pas ce qu'est une action « irréversible » ou ce que signifie « demander confirmation ». Il expose seulement un point d'interception générique (onBeforeExecute, §6.4) que chaque module peut implémenter à sa façon. C'est délibéré : figer dans le cœur d'AXIS le modèle de permission à cinq niveaux d'AURA (§14) interdirait à un autre projet d'y brancher sa propre politique de sécurité — contraire à l'objectif de réutilisation (§6.8).

- Le module AURA pour AXIS implémente ce hook en appelant le Policy Engine d'AURA (§14) : une action classée CONFIRM y reste bloquée en attente de confirmation, qu'elle soit tapée au clavier ou demandée en langage naturel (détail §6.7).
- AXIS CORE lui-même n'exécute jamais de commande arbitraire du système d'exploitation : il n'y a pas d'action shell.exec ou équivalent dans son cœur. Seules les actions explicitement déclarées par un module enregistré sont invocables. Un module malveillant ou mal conçu pourrait en théorie exposer une telle action — mais ce serait un choix explicite de ce module, jamais une capacité native d'AXIS.
- Un script (.axs) s'exécute commande par commande sous les mêmes règles que l'usage interactif : chaque commande repasse par le hook d'interception du module qui la porte.

6.7 Le module AURA pour AXIS

C'est ici, et seulement ici, que le lien avec AURA existe. Le module AURA :

- déclare comme espaces de commandes AXIS les connecteurs déjà définis en §7 à §11 (git, task, reminder, security, vision, etc.) — leurs manifestes ne changent pas, ils sont simplement exposés à AXIS en plus de la conversation ;
- implémente le hook d'interception (§6.6) en appelant le Policy Engine et l'Audit Logger d'AURA (§13.1, §14), avec la mention d'origine « axis » dans le journal (§12.4) ;

- ajoute les commandes propres à l'intégration : `axis.run_command`, `axis.run_script`, `axis.define_alias`, détaillées en tant que connecteur au §15.

Toute commande tapée qui cible un espace du module AURA (`git`, `task`, `security`...) respecte donc exactement les mêmes règles que la conversation (F-24) — pas parce qu'AXIS l'impose, mais parce que c'est le module AURA lui-même qui les applique.

6.8 Indépendance et réutilisation dans d'autres projets

AXIS n'a de sens que s'il reste utilisable sans AURA. Concrètement :

- AXIS CORE se distribue comme un paquet à part, avec sa propre CLI (`axis` dans un terminal système classique) — utilisable seul, sans fenêtre applicative, pour un script ou un projet qui n'a besoin d'aucune des capacités d'AURA.
- Un futur projet X adopterait AXIS en écrivant son propre module (§6.4) — un manifeste déclarant ses actions et, s'il le souhaite, son propre hook d'interception — sans jamais forker ni modifier le cœur d'AXIS.
- Une amélioration du langage ou du moteur d'exécution d'AXIS (nouvelle structure de contrôle, meilleure autocomplétion) profite à tous les projets qui l'utilisent en même temps, y compris AURA, sans effort de portage.
- AXIS se versionne indépendamment d'AURA (§6.11) : une mise à jour d'AURA n'oblige pas à changer de version d'AXIS, et inversement.

6.9 Ergonomie et découvrabilité

- `help` — liste les espaces de commandes disponibles, quel que soit le module dont ils proviennent.
- `help <espace>` — liste les actions d'un espace donné, avec leurs paramètres.
- Autocomplétion des espaces, actions et paramètres connus.
- Historique des commandes de la session, rappelable (flèches haut/bas) et consultable (`history`).
- Les messages d'erreur renvoient vers la commande `help` concernée plutôt que d'afficher une erreur brute non exploitable.

6.10 Intégration à AURA — écran interne

Dans AURA, AXIS s'affiche comme un écran interne de la fenêtre applicative (§13.2, §13.3), au même titre que la toile, la Cartographie ou l'Image Lab — jamais une fenêtre de terminal système séparée. C'est un choix d'intégration propre à AURA : rien n'empêche un autre projet d'embarquer AXIS différemment (CLI pure, plugin dans un autre outil). Maquette détaillée en §16.5.

6.11 Modèle de données

Entité	Portée	Rôle
Script	AXIS CORE	Fichier <code>.axs</code> : nom, chemin, contenu, date de dernière exécution — indépendant du module utilisé.
Alias	AXIS CORE	Nom court, commande cible, portée (session ou persistant).

CommandHistory	AXIS CORE	Commande tapée, horodatage, statut, résultat résumé.
ModuleManifest	AXIS CORE	Espace, actions déclarées, module d'origine, hook d'interception enregistré.

Les entités propres à AURA (Action/actions_log, §12.4) restent définies côté module AURA, pas côté AXIS CORE — AXIS ne journalise que ce qui concerne son propre fonctionnement (historique de commandes, scripts), pas la sémantique métier de chaque module.

6.12 AXIS et AURA AUTONOMY — un exemple bout en bout

Pour illustrer la cohérence architecturale recherchée : l'utilisateur écrit et teste un script `backup-projets.axs` interactivement dans AXIS jusqu'à ce qu'il fasse ce qu'il faut, en utilisant les commandes exposées par le module AURA, puis crée une règle AURA AUTONOMY (§5.9) « tous les jours à 20h, exécuter `axis.run_script(backup-projets.axs)` ». Le script écrit une fois devient ainsi une automatisation durable — AURA AUTONOMY appelle l'action `axis.run_script` du module AURA (§6.7, §15) sans jamais avoir besoin de connaître le fonctionnement interne d'AXIS CORE.

6.13 Évolutivité à long terme

Cette séparation en deux couches (§6.2) est ce qui rend l'architecture durable :

- Ajouter une capacité à AURA (nouveau connecteur, §7 à §11) l'expose automatiquement dans AXIS dès que le module AURA déclare son manifeste mis à jour — aucune modification d'AXIS CORE.
- Faire évoluer AXIS CORE (nouvelle syntaxe, meilleure gestion des erreurs) ne demande aucune modification du module AURA tant que l'interface de module (§6.4) reste stable.
- Un futur projet peut adopter AXIS sans jamais toucher au code d'AURA ni à celui d'AXIS CORE — il n'écrit que son propre module.
- Si AURA disparaissait, AXIS resterait un projet viable et utilisable par d'autres — et réciproquement, une évolution majeure d'AXIS ne remet pas en cause l'architecture d'AURA tant que le contrat de module est respecté.

7. Connecteur Développement

Statut : codé et testé (`git.status`, `git.commit`, `git.push` validés sur un dépôt réel).

Action	Sensibilité	Description
<code>git.status</code>	Lecture	Retourne l'état du dépôt courant.
<code>git.commit</code>	Réversible	Crée un commit local à partir des fichiers indiqués.
<code>git.push</code>	Irréversible	Publie les commits sur le dépôt distant — confirmation obligatoire.

`unity.build` retiré en V0.4 (lié au développement de jeux vidéo, hors du nouveau périmètre technique, §1.1). Le connecteur reste ouvert à l'ajout d'actions pour d'autres outils de développement (conteneurs, CI locale) au fil de l'usage réel, sans changer sa forme.

8. Connecteur Productivité

Statut : codé et testé.

Action	Sensibilité	Description
task.create	Réversible	Crée une tâche avec titre, échéance et priorité.
task.complete	Réversible	Marque une tâche comme terminée.
reminder.schedule	Réversible	Programme un rappel ponctuel ou récurrent.

9. Connecteur Cartographie

Statut : spécifié. Fond de carte OpenStreetMap ; rendu 2D uniquement, style noir/rouge dessiné par L. Cet écran vit dans la fenêtre applicative d'AURA (§13.2), pas dans une page web séparée.

Action	Sensibilité	Description
map.render_world	Lecture	Affiche la vue mondiale par défaut.
map.zoom_to	Lecture	Zoom vers un pays, une ville ou un lieu précis.
map.add_marker	Réversible	Ajoute un repère statique.
map.stream_markers	Réversible	Met à jour dynamiquement des repères en direct.

10. Vision avancée & AURA Image Enhancer

Sous-module d'AURA VISION dédié à l'amélioration, la restauration et l'agrandissement d'images. L'original n'est jamais écrasé par défaut.

10.1 Fonctions

Fonction	Description	Priorité
Upscaling	Augmenter la définition tout en préservant et reconstruisant les détails.	Haute
Sharpening	Renforcer la netteté de manière contrôlée.	Haute
Denoising	Réduire le bruit numérique.	Haute
Deblur	Atténuer certains flous lorsque le traitement est possible.	Moyenne
Restoration	Restaurer des photos anciennes, abîmées ou dégradées.	Moyenne
Color / Light	Correction de couleurs, contraste, luminosité et exposition.	Moyenne
Text Enhance	Améliorer la lisibilité de texte photographié (utile à la numérisation de documents techniques).	Moyenne

Face Detail retiré en V0.4 : capacité orientée portrait/personnel sans usage technique clair dans le périmètre recentré ; à réintroduire si un besoin réel apparaît.

10.2 Modes et parcours

- Mode automatique : AURA analyse le type d'image et ses défauts, propose une chaîne de traitement, produit un aperçu.
- Mode manuel : niveau d'upscaling, netteté, débruitage, lumière/couleurs configurables ; restauration activable.
- Interface AURA IMAGE LAB : zone originale/résultat, curseur de comparaison, boutons Aperçu/Améliorer/Exporter, réinitialisation — écran interne de la fenêtre applicative, pas une page web.

10.3 Connecteur Vision — actions image

Action	Sensibilité	Description
vision.analyze_image	Lecture	Analyse du contenu et de la qualité de l'image.
vision.enhance_image	Réversible	Produit une version améliorée sans modifier l'original.
vision.upscale_image	Réversible	Augmente la résolution.
vision.denoise_image	Réversible	Réduit le bruit et certains artefacts.
vision.sharpen_image	Réversible	Améliore la netteté.
vision.restore_photo	Réversible	Restaure une photographie dégradée.
vision.compare_images	Lecture	Compare l'original et le résultat.
vision.export_image	Réversible	Exporte le résultat vers un emplacement choisi.
vision.generate_image	Réversible	Génère une image à partir d'une description, distincte de l'amélioration.

10.4 Principes de qualité

- L'image originale n'est jamais écrasée par défaut.
- AURA distingue les détails réellement présents des détails reconstruits par un modèle.
- Un résultat généré peut être approximatif : jamais présenté comme une information certaine.
- Le traitement conserve le ratio d'image sauf demande explicite ; les traitements lourds affichent une progression.

11. Extensions transverses et agents

Cinq extensions greffées sur des modules existants.

11.1 Extensions

- Navigation web autonome — extension d'AURA OS (détail §5.4).
- Applications mobiles — extension d'AURA CODE (détail §5.6).

- Génération d'images — extension d'AURA VISION, distincte de l'Image Enhancer qui n'améliore que l'existant (détail §10.3).
- Vérification de faits — extension d'AURA RESEARCH : recoupement de sources avant de présenter une information comme fiable.
- Analyse de code et gestion des environnements de développement — extension d'AURA CODE (nouveau, V0.4, détail §5.6).

Montage audio/vidéo retiré en V0.4 : rattaché au pipeline multimédia dans la feuille de route V0.3.2, il perd sa justification directe une fois le périmètre recentré sur les domaines techniques ; à réévaluer si un besoin réel (ex. documentation vidéo technique) apparaît.

11.2 AURA EDUCATION

Fonction	Description	Priorité
Explications pédagogiques	Reformuler une notion technique (code, sécurité, système) à un niveau adapté à l'utilisateur.	Moyenne
Tutorat personnalisé	Suivi d'apprentissage dans la durée sur un sujet technique, appuyé sur la mémoire de projet.	Moyenne
Traduction de langues	Traduction et adaptation de texte technique entre langues.	Moyenne

Recentré en V0.4 sur la pédagogie technique (développement, sécurité, systèmes) plutôt que sur l'explication de sujets généraux, pour rester cohérent avec l'orientation du projet.

11.3 AURA OFFICE

Fonction	Description	Priorité
Word / Excel / PowerPoint	Création et édition de documents, tableurs et présentations — notamment rapports techniques et de sécurité.	Haute
PDF	Lecture, fusion, remplissage de formulaires, export.	Haute
Tableurs & modèles	Feuilles de calcul et modèles réutilisables (ex. suivi de vulnérabilités, inventaire d'infrastructure).	Moyenne

12. Modèle de données

La mémoire persistante d'AURA combine les entités de base (V0.2) et les entités ajoutées par l'extension V0.3. Les entités propres à AXIS (Script, Alias, CommandHistory, ModuleManifest) vivent dans le stockage d'AXIS CORE, pas dans celui d'AURA — détaillées en §6.11.

12.1 Entités de base (V0.2)

Entité	Rôle
Utilisateur	Profil unique (mono-utilisateur).
Préférence	Réglage mémorisé (style, outils, habitudes).

Projet	Contexte de travail suivi dans la durée.
Interaction	Historique des échanges.
Action	Journal des actions exécutées par un connecteur.
Connecteur	Registre des modules actifs et de leurs permissions.

12.2 Entités ajoutées (V0.3)

Entité	Rôle
Agent	Nom, version, spécialité, outils autorisés, statut.
Tool	Action exposée par un connecteur, paramètres, sensibilité.
Event	Événement détecté : type, source, priorité, horodatage, données.
Observation	Mesure issue d'un système autorisé.
Task	Tâche planifiée : étapes, état, priorité, déclencheur.
Policy	Règle d'autorisation ou de refus.
Prediction	Prévision : données sources, probabilité, confiance, date.
Alert	Alerte générée par AURA : niveau, cause, statut.
SystemMetric	Métrique CPU/GPU/RAM/disque/réseau/processus.

12.3 Exemple de table détaillée — actions_log

Champ	Type	Description
id	INTEGER (PK)	Identifiant unique.
connecteur_id	INTEGER (FK)	Connecteur à l'origine de l'action.
type_action	TEXT	Ex. « git.commit », « axis.run_script ».
origine	TEXT	conversation / axis / autonomy.
sensibilité	TEXT	lecture / réversible / irréversible.
statut	TEXT	exécuté / confirmé / refusé / échoué.
date	DATETIME	Horodatage de l'action.
détails	JSON	Paramètres et résultat de l'action.

Champ « origine » ajouté en V0.4 pour distinguer une action déclenchée en conversation, depuis AXIS (via le module AURA), ou par une règle AURA AUTONOMY — sans quoi le journal ne permettrait plus de répondre à « qui/quoi a fait ça » une fois AXIS intégré. Cette table reste propre à AURA (§12.2) ; le journal des commandes AXIS elles-mêmes (CommandHistory) est décrit en §6.11.

13. Architecture technique

13.1 Vue d'ensemble

L'architecture V0.2 séparait déjà le cœur IA, l'orchestrateur, les connecteurs, la mémoire et les interfaces. L'extension V0.3 a ajouté un gestionnaire d'agents, un moteur d'événements, un moteur de politiques et une couche d'observation. La V0.4 intègre AXIS comme deuxième point d'entrée utilisateur, en parallèle de la fenêtre de conversation — mais, à la différence des autres briques, AXIS CORE est représenté **hors** du périmètre d'AURA : c'est un projet externe qu'AURA embarque et relie à son orchestrateur via le module AURA pour AXIS (§6.7), sans réécrire ni l'un ni l'autre.



AURA STREAM et AURA GAME retirés du schéma (périmètre Gaming & Streaming supprimé, §1.1). AXIS CORE apparaît en dehors du bloc AURA pour marquer visuellement son indépendance (§6.1) : le module AURA pour AXIS est le seul pont entre les deux, exactement comme le serait le module d'un autre projet hôte. Une commande AXIS qui cible le module AURA ne traite rien elle-même dans AXIS CORE ; elle est transmise au Tool Manager via ce module, sans passer par le Reasoner/Planner d'AURA CORE — c'est la différence de fond avec la conversation, plus lente mais capable d'interpréter une demande ambiguë.

13.2 Stack technique

Composant	Technologie	Statut réel
Cœur IA	API Claude (Anthropic)	En service
Backend / orchestrateur	Node.js + Express	En service
Lanceur	Commande CLI tapée dans un terminal système quelconque	Spécifié
Fenêtre applicative	Electron ou Tauri — rendu HTML/CSS/JS en fenêtre native ; jamais un site accessible par navigateur ou URL	Spécifié
AXIS CORE	Paquet Node.js indépendant (parseur du langage, moteur d'exécution, registre de modules) — pas une dépendance d'AURA, une dépendance qu'AURA embarque (§6.1)	Spécifié
Module AURA pour AXIS	Bridge : expose les connecteurs §7-§11 comme modules AXIS, branché sur le Tool Manager et le Policy Engine (§13.1, §14)	Spécifié
Mémoire / base de données	SQLite via node:sqlite	En service — choisi pour éviter la compilation native
Cartographie	Leaflet + OpenStreetMap, vue 2D, écran interne de la fenêtre applicative	Spécifié
Identité visuelle	Fond noir, accents rouges, grille en pointillés, HUD circulaire	Spécifié — à porter à la fenêtre applicative

Connecteur gaming (API Riot Games / Steam) et connecteur streaming (OBS WebSocket) retirés en V0.4.

13.3 Identité visuelle

Fond noir (proche du #050505), traits fins rouges/blancs pour les tracés, grille en pointillés en arrière-plan, badges HUD circulaires animés façon JARVIS — voir aussi l'addendum V0.3.3 (typographie IBM Plex Mono, palette noir #050505 / rouge #A51706) qui reste la référence identité de ce document. Cette direction s'applique à l'ensemble des écrans de la fenêtre applicative, y compris la cartographie, l'Image Lab et l'écran AXIS — cette habillage visuel est propre à l'intégration AURA, il n'est pas porté par AXIS CORE lui-même (§6.10).

AURA n'a aucune présence tant qu'elle n'est pas lancée (F-12, F-22) : ni icône, ni processus, ni page web. Une commande tapée dans un terminal système ouvre une fenêtre d'application dédiée (Electron ou Tauri) — c'est cette fenêtre, et elle seule, qui constitue AURA. Elle affiche un rendu en toile : des nœuds représentant les agents (branches principales) et leurs outils (branches secondaires), reliés entre eux par des connexions qui correspondent à de vraies relations (§5.6.1) et s'illuminent en rouge selon l'activité,

dans l'esprit visuel de The Machine (Person of Interest). La zone de conversation, la toile, l'Image Lab, la Cartographie et l'écran AXIS sont des écrans internes de cette même fenêtre — il n'existe aucune interface web séparée.

13.4 Portée et empreinte système

- Aucun processus, service ou tâche planifiée ne s'exécute avant la commande de lancement (F-22).
- Aucune icône permanente dans la barre des tâches, le menu Démarrer ou le dock.
- La fenêtre applicative se ferme et libère toute ressource quand l'utilisateur quitte AURA — y compris l'instance d'AXIS embarquée et tout script AXIS en cours.
- Conséquence assumée : AURA AUTONOMY (§5.9) ne surveille que pendant qu'une session est ouverte, pas 24 h/24, et un script planifié via Autonomy ne s'exécute donc que si AURA est lancée.

14. Système de permissions

Deux échelles complémentaires coexistent : la sensibilité d'une action (utilisée dans chaque contrat de connecteur) et le niveau de permission accordé au module qui la porte (utilisé par le moteur de politiques de l'orchestrateur). Les deux s'appliquent identiquement, que l'action soit déclenchée par la conversation, par une règle AURA AUTONOMY ou par une commande AXIS ciblant le module AURA (§6.6, §6.7, F-24) — AXIS CORE lui-même ne connaît pas ces règles, c'est le module AURA qui les applique.

14.1 Sensibilité d'action (V0.2)

- Lecture — aucune conséquence, exécutée librement.
- Réversible — effet local qui peut être annulé, exécutée librement sauf si le contexte l'exige.
- Irréversible — confirmation explicite obligatoire avant exécution.

14.2 Niveaux de permission (V0.3)

Niveau	Nom	Principe
0	OBSERVE	Lecture / observation uniquement.
1	SUGGEST	Analyse et proposition sans exécution.
2	AUTO	Action sûre, locale et réversible autorisée automatiquement.
3	CONFIRM	Confirmation explicite avant exécution.
4	LOCKED	Action interdite au module.

Toute action financière, suppression définitive, envoi externe, modification critique ou accès à une donnée sensible reste soumise à confirmation explicite (niveau CONFIRM), quel que soit le module qui la déclenche et quelle que soit l'interface utilisée (conversation ou AXIS).

15. Contrats de connecteurs — synthèse étendue

Chaque connecteur expose un manifeste uniforme : action, sensibilité, scope requis, entrée, sortie (détaillé §7 à §10 pour les connecteurs codés ou entièrement spécifiés, §6.7 pour les actions propres au module AURA pour AXIS). Les modules V0.3/V0.4 ajoutent les familles d'actions suivantes.

Connecteur	Exemples d'actions
System	system.metrics, process.list, app.launch, app.close, network.diagnose
Vision	vision.capture_screen, vision.ocr, vision.analyze_image
Voice	voice.listen, voice.speak, voice.stop
Analytics	analytics.query, analytics.detect_anomaly, analytics.forecast, analytics.predict
Automation	task.create_workflow, task.schedule, task.pause
Security	security.audit_project, security.scan_logs, security.check_dependencies, security.pentest_scan
Agents	agent.start, agent.stop, agent.delegate
Module AURA pour AXIS	axis.run_command, axis.run_script, axis.define_alias, axis.list_commands (détail §6.7) — actions ajoutées par AURA, pas par AXIS CORE

16. Maquettes et parcours utilisateur

Tous les écrans ci-dessous sont des écrans internes de la même fenêtre applicative (§13.2, §13.3) — il n'existe ni page web, ni fenêtre séparée.

16.1 Écran principal — toile & conversation

- Toile en arrière-plan : nœuds représentant les agents (branches principales) et leurs outils (branches secondaires), reliés par des connexions qui correspondent à de vraies relations (§5.6.1) et s'illuminent en rouge selon l'activité.
- Zone de conversation superposée en bas d'écran : fil de dialogue, champ de saisie fixe — sur le même écran que la toile, jamais un écran séparé (F-21).
- Panneau latéral rétractable : projets actifs, accès rapide aux modules.
- Panneau de contexte rétractable : préférences appliquées, dernières actions du journal.

16.2 Écran mémoire & préférences

- Liste des préférences par catégorie, édition et suppression individuelle.
- Bouton « Effacer tout », protégé par confirmation explicite ; historique des projets.

16.3 Écran carte interactive (AURA WORLD)

- Écran interne de la fenêtre applicative — vue exclusivement plane 2D ; vue mondiale par défaut, zoom sur demande.

- Fond noir, réseau routier en fines lignes rouges/blanches ; badges HUD, grille en pointillés.

16.4 Écran AURA IMAGE LAB

- Écran interne de la fenêtre applicative — zone Originale / Zone Résultat, curseur de comparaison avant/après.
- Boutons Aperçu, Améliorer, Exporter ; réinitialisation vers l'original.

16.5 Écran AXIS (nouveau, V0.4)

- Écran interne de la fenêtre applicative, habillé dans le même esprit visuel que le reste (fond noir, texte en IBM Plex Mono, accents rouges) — un habillage propre à l'intégration AURA, pas une caractéristique d'AXIS CORE lui-même (§6.10).
- Zone de sortie (haut) : résultat des commandes, défilant, avec code couleur par statut (succès, confirmation en attente, échec) — voir aussi le panneau Activité (§16.1), alimenté par le module AURA pour AXIS de la même façon que les autres connecteurs.
- Ligne de saisie (bas) : commande courante, autocomplétion en surimpression, historique accessible.
- Un indicateur visuel distingue une commande en attente de confirmation (§14.2, niveau CONFIRM, appliquée par le module AURA) des commandes déjà exécutées.
- Bouton d'accès à un script en cours d'édition (ouverture d'un éditeur simple pour les fichiers .axs), avec bouton Exécuter.

17. Exigences non fonctionnelles

Catégorie	Exigence
Performance	Temps de réponse cible inférieur à 3 secondes pour une requête conversationnelle simple ; inférieur à 500 ms pour une commande AXIS sur une action locale.
Sécurité	Aucun identifiant, mot de passe ou moyen de paiement stocké en clair.
Confidentialité	Données mémorisées locales ou hébergées sur une infrastructure choisie par l'utilisateur.
Fiabilité	La panne d'un module ou d'un agent n'empêche pas le fonctionnement des autres, ni celui d'AXIS pour les modules restés chargés.
Évolutivité	Un nouveau connecteur AURA s'ajoute sans réécrire l'orchestrateur ni AXIS CORE, et devient automatiquement disponible via le module AURA pour AXIS (§6.4, §6.13).
Accessibilité	Interface utilisable en français, lisible dans la fenêtre applicative.
Portée d'accès	Usage strictement local pour le MVP ; accès distant en extension ultérieure.
Empreinte système	Aucun processus ni icône persistants ; AURA n'existe que pendant une session lancée depuis le terminal système (F-22).
Réutilisabilité	AXIS CORE ne doit contenir aucune référence au code, à la mémoire ou aux connecteurs d'AURA — vérifiable en confirmant qu'AXIS peut se lancer et exécuter un module minimal sans le module AURA installé (§6.8).

18. Contraintes, limites et principes de sécurité

18.1 Limites techniques réalistes

Aucune intelligence artificielle actuelle ne dispose d'un accès illimité à tous les systèmes numériques : chaque capacité d'AURA doit être explicitement construite via un connecteur, avec les autorisations correspondantes. AURA ne peut agir que sur ce à quoi elle est effectivement reliée — et cela vaut identiquement pour une commande AXIS qui cible le module AURA.

18.2 Contraintes éthiques et légales

- Aucune action financière sans confirmation explicite et individuelle.
- Aucun contournement des conditions d'utilisation de services tiers.
- Cybersécurité : le volet défensif (audit, dépendances, secrets, journaux) s'applique aux projets et systèmes autorisés. Le volet offensif (pentest) est strictement limité aux systèmes propres de l'utilisateur, aux environnements CTF, ou aux cibles couvertes par une autorisation écrite explicite (bug bounty) — jamais à un système tiers non autorisé, jamais pour compromettre un tiers.
- **AXIS CORE n'expose aucun interpréteur système générique** — seules les actions déclarées par un module enregistré sont invocables (F-24), et le module AURA n'en déclare aucune de ce type ; AXIS ne peut donc pas servir à contourner les règles ci-dessus par un détournement technique.
- Toute donnée personnelle traitée par AURA reste sous le contrôle de l'utilisateur, avec possibilité de suppression complète.

18.3 Limites assumées du projet

Le nom et l'ambition « omnipotence » désignent une trajectoire d'extension continue, pas une capacité initiale ni un accès illimité de fait. La référence à JARVIS et à la Machine sert de direction fonctionnelle et esthétique, pas de promesse littérale : chaque capacité dépend d'un connecteur, d'une autorisation et d'un système réellement accessible. Chaque édition de ce cahier des charges couvre un périmètre volontairement délimité ; l'historique des versions figure en section 24.

19. Scénarios de test et critères d'acceptation

Exigence / Scénario	Test	Critère d'acceptation
F-01 — Compréhension	Question courante en français.	Réponse cohérente en moins de 3 secondes.
F-03 — Routage	« Vérifie les dépendances de ce projet ».	Routage automatique vers AURA SECURITY.
F-06 — Mémoire	« Qu'est-ce que tu sais de moi ? » puis « efface tout ».	Liste puis suppression définitive après confirmation.
F-09 — Confirmation	Demande d'envoi d'un message Discord.	Aperçu présenté, envoi bloqué sans confirmation.
F-12 / F-21 — Lancement	Commande de lancement tapée dans un terminal système.	Une fenêtre d'application dédiée s'ouvre avec le rendu toile ; conversation et toile sur le même écran.

F-22 — Empreinte système	Vérification des processus avant toute commande de lancement.	Aucun processus, service ou icône AURA détecté tant que la commande n'a pas été exécutée.
F-23/24 — AXIS	Commande <code>git.push</code> tapée directement dans AXIS (module AURA).	Aperçu présenté, action bloquée sans confirmation — comportement identique à la conversation.
F-25 — Script	Script <code>.axs</code> combinant <code>task.create</code> et <code>reminder.schedule</code>, lancé via <code>run</code>.	Les deux actions s'exécutent dans l'ordre écrit, chacune journalisée avec origine « <code>axis</code> ».
§6.8 — Indépendance	Lancer AXIS CORE en CLI autonome, sans le module AURA installé, avec un module minimal de test.	AXIS démarre et exécute les commandes du module de test sans erreur liée à AURA — aucune dépendance cachée.
AURA SYSTEM — Diagnostic	« Vérifie mon PC. »	Collecte de métriques, analyse d'anomalies, rapport.
AURA SECURITY — Pentest	Scan lancé sur un système non autorisé (conversation ou AXIS).	Action refusée, aucune exécution, dans les deux cas.
AURA AUTONOMY — Simulation	Règle déclenchée en mode simulation.	Action décrite mais non exécutée.
Image Enhancer	« Améliore cette photo en 4K. »	Analyse, traitement, aperçu avant export, original intact.

20. Feuille de route et estimation de charge

Phase	Contenu	Charge / statut
MVP	Cœur conversationnel + mémoire de base + modèle de données initial.	Fait
V1 fondations	Connecteur Développement + connecteur Productivité.	Fait
V0.3-A	Refonte de l'orchestrateur : agents, outils, permissions, événements, journalisation.	À faire
V0.3-B	AURA OS + System Monitor + automatisations locales + navigation web autonome.	À faire
V0.3-C	Voice + Vision + première fenêtre applicative (toile).	À faire
V0.3-D	Analytics + anomalies + prévisions probabilistes / IA-ML.	À faire
V0.4-a — AXIS CORE	Développement d'AXIS comme paquet indépendant (dépôt séparé) : parseur du langage, moteur d'exécution, registre de modules, CLI autonome. Aucune référence à AURA à ce stade (§6.1 à §6.6, §6.8).	À faire — priorité immédiate après la toile (§13.3)
V0.4-b — Module AURA pour AXIS	Bridge exposant les connecteurs existants (§7-§11) comme modules AXIS, branché sur le Policy Engine et l'Audit Logger d'AURA (§6.7). Agents consolidés : analyse de code, administration réseau (AURA CODE, AURA OS). Vérification de faits, AURA EDUCATION recentrée sur la pédagogie technique.	À faire — dépend de V0.4-a

V0.5	AURA OFFICE ; pipeline multimédia résiduel à réévaluer selon besoin réel (§11.1).	À faire
Fenêtre applicative	Lanceur terminal + fenêtre Electron/Tauri, toile maillée, zéro empreinte hors session (F-12, F-21, F-22).	À faire
V1	Plateforme AURA intégrée : mémoire, voix, vision, développement, système, sécurité, AXIS, analytics, automatisation.	À faire
V2+	Extension multi-appareils, domotique, nouveaux connecteurs et capacités spécialisées.	À faire

Les phases nommées « V0.3-x », « V0.4 », « V0.5 » etc. désignent des jalons de développement internes à la feuille de route, distincts du numéro de version de ce document (« Cahier des charges V0.4 ») — une distinction déjà présente dans les éditions précédentes.

21. Budget prévisionnel

Estimation indicative pour un usage personnel ; les modules ajoutés en V0.3+ (voix, vision, sécurité) demanderont leur propre estimation une fois les fournisseurs choisis.

Poste	Estimation	Remarque
API Claude (usage mensuel)	≈ 10 à 40 € / mois	Varie selon le volume de requêtes et la taille du contexte.
Hébergement backend	0 €	Exécution strictement locale — pas de service hébergé nécessaire.
Base de données	0 €	SQLite local (node:sqlite) pour le MVP.
Fenêtre applicative	0 €	Electron ou Tauri sont libres ; pas de nom de domaine ni d'hébergement puisqu'il n'y a pas de web.
Fournisseur cartographique	0 €	Données OpenStreetMap libres, style personnalisé.
Assets graphiques	Variable	Selon les besoins ponctuels.

Les postes liés au connecteur gaming (API Riot Games/Steam) et au connecteur streaming (OBS WebSocket), déjà à 0 € dans la V0.3.2, sont retirés puisque ces connecteurs n'existent plus.

22. Critères de succès

- AURA est utilisée quotidiennement pour au moins un module prioritaire.
- Le temps consacré aux tâches techniques répétitives diminue mesurablement — **AXIS permet d'accomplir en une commande ou un script ce qui prendrait plusieurs échanges en conversation.**
- Chaque nouveau connecteur AURA s'intègre sans régression sur les modules existants, et devient automatiquement utilisable depuis AXIS sans travail d'intégration séparé (§6.4).

- L'utilisateur garde une confiance totale : aucune action sensible sans son accord, quelle que soit l'interface utilisée pour la déclencher.
- AURA choisit automatiquement le ou les agents nécessaires à une demande en conversation.
- AURA peut observer un système autorisé, analyser les données et produire un diagnostic compréhensible.
- Un module peut être arrêté sans arrêter le reste du système.
- AURA peut fonctionner en mode simulation avant l'activation de l'autonomie.
- AURA peut combiner plusieurs domaines dans une seule tâche, ou dans un seul script AXIS.
- AURA ne laisse aucune trace (processus, icône) tant qu'elle n'a pas été lancée depuis le terminal système.
- **AXIS CORE fonctionne de façon autonome, sans le module AURA** — la preuve concrète que le Terminal n'est pas propriétaire (§6.8, §17).

23. Risques et mitigation

Risque	Impact	Mitigation
Périmètre trop large, projet jamais terminé	Élevé	Développement modulaire, priorisation par phase (§20), retrait explicite des domaines hors périmètre (§1.1).
Dépendance à un fournisseur d'API unique	Moyen	Cœur IA isolé des connecteurs, remplaçable.
Action non désirée exécutée automatiquement	Élevé	Confirmation obligatoire sur toute action sensible (§14), identique en conversation et en AXIS.
Fuite ou perte de données personnelles	Élevé	Stockage local maîtrisé, droit d'effacement total.
AXIS donne un faux sentiment de puissance illimitée	Moyen	AXIS CORE n'expose aucun interpréteur système générique ; le module AURA applique les mêmes garde-fous que les autres interfaces (§6.6, F-24) ; documentation claire de la limite dès l'écran d'aide (§6.9).
Le module AURA se désynchronise d'AXIS CORE (versions incompatibles)	Moyen	Contrat de module versionné explicitement (§6.4, §6.13) ; tests d'intégration entre les deux avant chaque mise à jour d'AXIS.
Pentest utilisé hors périmètre autorisé	Élevé	Restriction stricte à systèmes propres / CTF / autorisation écrite (§5.8, §18.2), appliquée identiquement depuis AXIS.
Surveillance Autonomy perçue comme un service caché	Moyen	Empreinte système nulle hors session, documentée explicitement (F-22, §13.4).
Dérive de version : documents qui divergent du code réel	Moyen	Ce cahier des charges + historique des versions (§24) comme référence unique.

24. Historique des versions

Version	Contenu ajouté
V0.2	Cahier des charges initial : cœur conversationnel, mémoire, connecteurs Productivité/Développement/Gaming/Streaming/Cartographie, identité noir/cyan.
V0.3	Extension « JARVIS + The Machine » : CORE avancé, VOICE, VISION, OS, ANALYTICS, AGENTS, SYSTEM MONITOR, SECURITY, AUTONOMY, WORLD, système de permissions à 5 niveaux.
V0.3.1	Sous-module AURA IMAGE ENHANCER au sein d'AURA VISION.
V0.3.2	Intégration de la carte mentale des capacités : navigation web autonome, apps mobiles, génération d'images, vérification de faits, AURA EDUCATION, AURA OFFICE ; confirmation du périmètre pentest encadré ; passage à l'identité noir/rouge.
V0.3.3	Identité visuelle définitive : typographie IBM Plex Mono, palette noir #050505 / rouge #A51706 (addenda et correctif).
V0.4	Suppression complète du périmètre Gaming & Streaming (connecteur §8 V0.3.2, agents AURA GAME/AURA STREAM, écran dédié, toute mention associée). Recentrage du projet sur les domaines informatiques et techniques (développement, cybersécurité, administration système/réseau, automatisation, données, IA) — agents existants étoffés (AURA CODE, AURA OS, AURA ANALYTICS) plutôt que multipliés. Réseau d'agents : 12 → 10.
V0.4 (révision)	Introduction d'AXIS : le Terminal n'est plus une fonctionnalité propriétaire d'AURA mais un projet indépendant et réutilisable (§6), avec une architecture en deux couches — AXIS CORE (langage, moteur d'exécution, système de modules génériques, aucune référence à AURA) et le module AURA pour AXIS (bridge qui expose les connecteurs d'AURA et applique son Policy Engine). Scripts renommés .asc → .axs. Exigences F-23 à F-25 révisées en conséquence. Mise à jour correspondante de la carte mentale, avec séparation visuelle explicite entre la branche AXIS (cœur indépendant) et le module AURA (§25).

25. Annexe — Glossaire

Cœur IA

Composant central basé sur un modèle de langage, responsable de la compréhension et du raisonnement.

Connecteur

Module indépendant reliant AURA à un service ou domaine précis.

Agent

Module spécialisé délégué par AURA CORE pour un domaine (ex. CODE, SECURITY, EDUCATION).

Orchestrateur

Service qui reçoit une requête, choisit les connecteurs/agents à solliciter et applique les règles de permission.

Policy Engine

Composant qui évalue le niveau de permission requis pour une action donnée.

Event Engine

Composant qui détecte et route les événements déclenchant une action de l'Autonomy.

MVP

Minimum Viable Product, première version fonctionnelle couvrant le strict nécessaire.

Action irréversible

Action dont les effets ne peuvent pas être annulés simplement.

OBSERVE / SUGGEST / AUTO / CONFIRM / LOCKED

Les cinq niveaux de permission du moteur de politiques (détail §14.2).

Fenêtre applicative

Fenêtre d'application dédiée (Electron ou Tauri) ouverte par une commande terminal — l'unique surface d'AURA, jamais un site web (§13.2, F-12).

Toile

Rendu graphique de la fenêtre applicative où agents (branches principales) et outils (branches secondaires) apparaissent comme des nœuds reliés par des connexions représentant de vraies relations — inspiré de The Machine (Person of Interest), voir §5.6.1 et §16.1.

AXIS

Terminal générique, indépendant et réutilisable — langage textuel (espace.action, pipes, variables, scripts .axs), système de modules génériques. Ne connaît pas AURA ; se développe et se distribue comme un projet à part (§6.1 à §6.6, §6.8, §6.13).

Module AXIS

Paquet qui s'enregistre auprès d'AXIS CORE en déclarant un manifeste (espaces de commandes, actions, hook d'interception) — un projet peut écrire son propre module sans modifier AXIS CORE (§6.4).

Module AURA pour AXIS

Le module qu'AURA fournit à AXIS : expose ses connecteurs existants (§7-§11) comme commandes AXIS et applique le Policy Engine et l'Audit Logger d'AURA via le hook d'interception (§6.7).

Script AXIS (.axs)

Fichier texte contenant une suite de commandes du langage AXIS, sauvegardé et réexécutable, indépendant du module utilisé (§6.5).

Espace de commande

Préfixe d'une commande AXIS identifiant le module ciblé (ex. « git » dans git.status, fourni par le module AURA) — voir §6.3.

Repère (marker)

Point d'intérêt affiché sur une carte, associé à des coordonnées géographiques.